



TESTE DE SISTEMAS TBIT MANAGER

Prof. Roselene Fenske



**KAIO MAZZA
EBERTH RODRIGUES
CAIO AIRES
DANIEL BALERA**

Prof. Roselene Fenske

Objetivo



Apresentar o planejamento e a execução dos testes realizados no sistema de gerenciamento de estoque, com foco em:

- Explicar os critérios utilizados para validar as funcionalidades do sistema
- Compartilhar os **resultados obtidos** durante a execução dos testes manuais
- Demonstrar a metodologia aplicada na criação e organização dos **60 casos de teste**
- Exibir um exemplo prático de **automação de teste com Selenium**, destacando seus benefícios
- Evidenciar a importância dos testes para garantir a qualidade e confiabilidade do sistema

Sistema testado



Nome: TBIT Manager

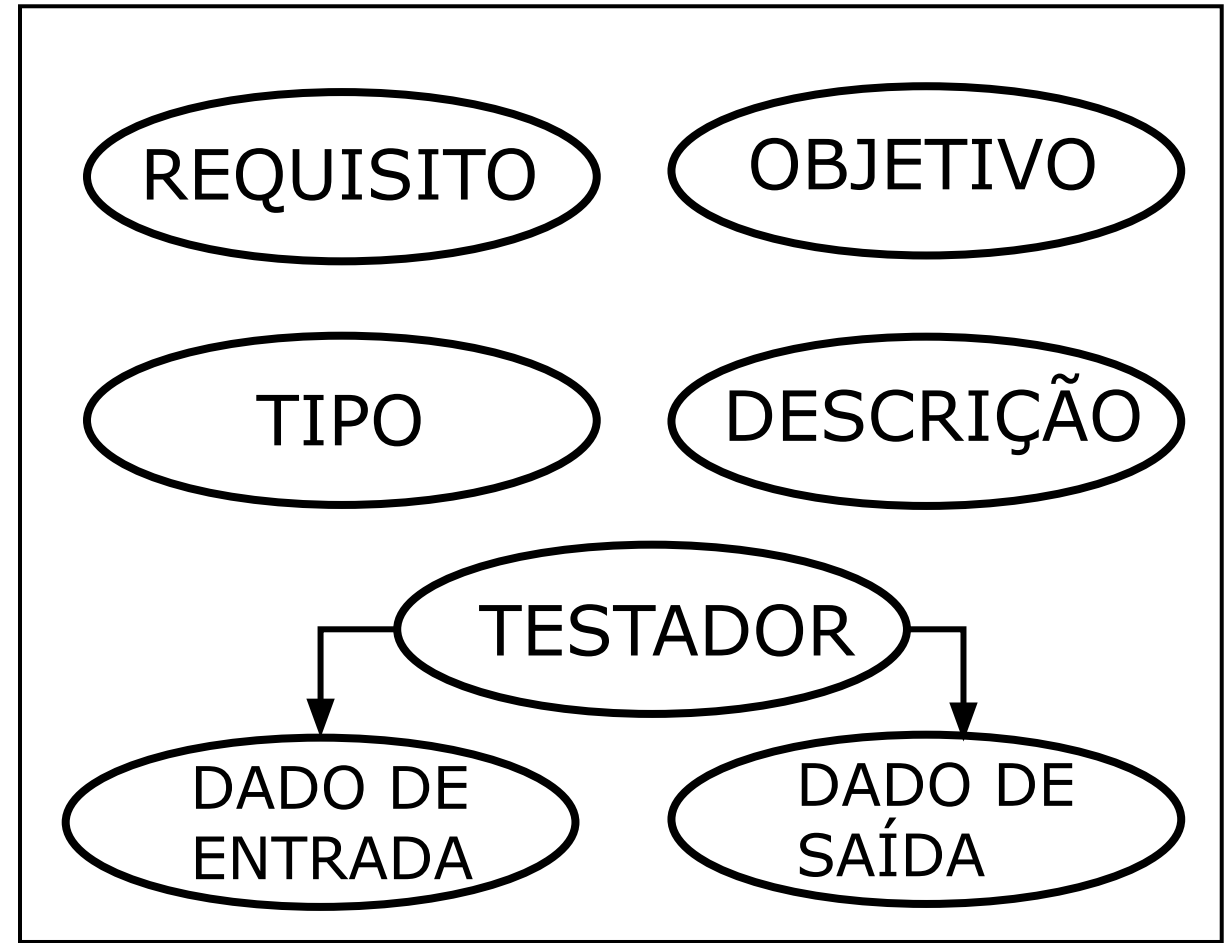
Finalidade: Sistema de gerenciamento de estoque voltado para controle de hardware

Principais funcionalidades:

- Cadastro, edição, exclusão e consulta de itens (CRUD padrão)
- Controle de entrada e saída de equipamentos
- Organização por categorias e localização física
- Geração de relatórios de movimentação e inventário



Estrutura dos testes



Metodologia



Foram realizados **60 testes** no total, abrangendo diferentes tipos para garantir a qualidade do sistema TBIT Manager:

- Funcionais
- Unitários
- Integração
- Regressão
- Performance
- Segurança
- Compatibilidade
- Usabilidade

Os testes foram organizados e distribuídos conforme um **cronograma de execução**, garantindo cobertura progressiva das funcionalidades e controle dos prazos.

Cronograma de testes			
Responsável	Data de início prevista	Data de real	Desc. Resultado
Eberth Rodrigues	02/09/25	01/09/25	Acesso permitido com credenciais corretas.
Eberth Rodrigues	02/09/25	01/09/25	Mensagem de erro exibida ao digitar senha incorreta.
Eberth Rodrigues	02/09/25	01/09/25	Formulário salvo com dados corretos.
Eberth Rodrigues	02/09/25	01/09/25	Sistema bloqueia salvamento e exibe alerta.
Eberth Rodrigues	05/09/25	01/09/25	Estoque atualizado com as 10 unidades adicionadas.
Eberth Rodrigues	05/09/25	01/09/25	Estoque reduzido em 5 unidades com sucesso.
Eberth Rodrigues	05/09/25	01/09/25	Sistema impede retirada e exibe mensagem de erro.
Eberth Rodrigues	05/09/25	01/09/25	Valor do estoque exibido corretamente pelo nome.

Automatização de testes



Objetivo do teste: Validar o cadastro de um funcionário no sistema **TBIT Manager**, garantindo que os dados inseridos sejam corretamente exibidos na tabela de registros.

Ferramenta utilizada: Selenium WebDriver com linguagem **Python**

Fluxo automatizado:

- Abertura do navegador e acesso ao formulário de cadastro
- Preenchimento dos campos obrigatórios com dados do produto
- Submissão do formulário
- Verificação da presença do produto na tabela de registros

Critério de sucesso: O produto deve aparecer na tabela com todos os dados preenchidos corretamente.

Resultado esperado: Mensagem de sucesso exibida e produto visível na tabela após o envio.

Benefícios da automação:

- Agilidade na execução de testes repetitivos
- Redução de erros humanos
- Facilidade na revalidação após correções

Automatização de testes



```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC
import time

# Função para teste automatizado
def
selenium_test(driver, nome, data_n, data_a, cpf, cidade, uf, telefone, email, usua
rio, senha, perfil):

    # 1. Abre a página do app de estoque
    driver.get(
"http://localhost:8080/Projeto-tbit-manager-estoque/Selenium/criar.php")

    driver.implicitly_wait(3)

    # 2. Encontra os campos de input e preenche com os dados do produto
    campo_nome = driver.find_element(By.ID, "nome_completo")
    campo_data_n = driver.find_element(By.ID, "data_nascimento")
    campo_data_a = driver.find_element(By.ID, "data_admissao")
    campo_cpf = driver.find_element(By.ID, "cpf")
    campo_cidade = driver.find_element(By.ID, "cidade")
    campo_uf = driver.find_element(By.ID, "uf")
    campo_telefone = driver.find_element(By.ID, "telefone")
    campo_email = driver.find_element(By.ID, "email")
    campo_usuario = driver.find_element(By.ID, "usuario")
    campo_senha = driver.find_element(By.ID, "senha")
    campo_perfil = driver.find_element(By.ID, "perfil")

    botao_cadastrar = driver.find_element(By.ID, "cadastrar_usuario")

    campo_nome.send_keys(nome)
    campo_data_n.send_keys(data_n)
    campo_data_a.send_keys(data_a)
    campo_cpf.send_keys(cpf)
    campo_cidade.send_keys(cidade)
    campo_uf.send_keys(uf)
    campo_telefone.send_keys(telefone)
    campo_email.send_keys(email)
    campo_usuario.send_keys(usuario)
    campo_senha.send_keys(senha)
    campo_perfil.send_keys(perfil)

    # 3. Clica no botão para adicionar
    time.sleep(5)
    botao_cadastrar.click()
    time.sleep(2) # Pausa para a pagina atualizar
```

```
# 4. Acessa a página onde a tabela está
driver.get(
"http://localhost:8080/Projeto-tbit-manager-estoque/Selenium/listar.php")

# 5. Espera a tabela aparecer
time.sleep(3)

data_n_invert = inverter_data(data_n)
data_a_invert = inverter_data(data_a)

# 6. Verifica se os dados do funcionário aparecem
tabela = driver.find_element(By.ID, "tabela_funcionarios")
dados_esperados = [
    nome, data_n_invert, data_a_invert, cpf,
    cidade, uf, telefone, email,
    usuario, senha, perfil
]

if all(dado in tabela.text for dado in dados_esperados):
    return print("Teste de cadastro de funcionário: SUCESSO!")
else:
    return print("Teste de cadastro de funcionário: FALHA!")

# Função para inverter datas
def inverter_data(data):
    test_date = data.split("-")
    return test_date[2]+"-"+test_date[1]+"-"+test_date[0]

# Entradas que vão ser testadas:
nome_completo = "Caio Aires"
data_nascimento = "30-01-2008"
data_admissao = "17-02-2025"
cpf = "000.000.000-00"
cidade = "Joinville"
uf = "SC"
telefone = "4002-8922"
email = "caio@gmail.com"
usuario = "caio_aires"
senha = "1234"
perfil = "ADM"

# Inicia o serviço do ChromeDriver
servico = webdriver.ChromeService(executable_path=
"C:/xampp/htdocs/Projeto-tbit-manager-estoque/chromedriver-win64/chromedriver.exe")
driver = webdriver.Chrome(service=servico)

try:
    selenium_test(driver, nome_completo, data_nascimento, data_admissao, cpf, cidad
e, uf, telefone, email, usuario, senha, perfil)

finally:
    driver.quit()
```


Resultados obtidos



Planejamento de testes					
Matriz de rastreabilidade de testes					
Objetivo do teste	Tipo de teste	Descrição	Estratégia do teste	Ambiente de teste	Ferramentas
Verificar login com credenciais válidas	Funcional	Testar se o usuário consegue acessar com login e senha corretos	Manual	Visual Studio Code	N/A
Verificar falha no login com senha errada	Funcional	Validar exibição de mensagem de erro ao digitar senha errada	Manual	Visual Studio Code	N/A
Cadastrar novo produto	Funcional	Preencher formulário com dados válidos e salvar	Manual	Visual Studio Code	N/A
Impedir cadastro com campos vazios	Funcional	Deixar campos obrigatórios em branco e tentar salvar	Manual	Visual Studio Code	N/A
Registrar entrada de estoque válida	Funcional	Adicionar 10 unidades ao estoque de um produto existente	Manual	Visual Studio Code	N/A
Registrar saída de estoque com quantidade válida	Funcional	Reduzir 5 unidades de produto com estoque suficiente	Manual	Visual Studio Code	N/A
Bloquear saída com estoque insuficiente	Funcional	Tentar retirar 100 unidades de um produto com estoque menor	Manual	Visual Studio Code	N/A
Consultar estoque atual	Funcional	Verificar valor do estoque pelo nome do produto	Manual	Visual Studio Code	N/A
Editar dados de produto	Funcional	Alterar nome e categoria de produto existente	Manual	Visual Studio Code	N/A
Excluir produto	Funcional	Deletar produto da base de dados do estoque	Manual	Visual Studio Code	N/A
Cadastro de fornecedor	Funcional	Preencher formulário com dados válidos e salvar novo fornecedor	Manual	Visual Studio Code	N/A
Edição de fornecedor	Funcional	Alterar dados de fornecedor existente e confirmar atualização	Manual	Visual Studio Code	N/A
Cadastro de categoria de produto	Funcional	Criar nova categoria com nome e descrição válidos	Manual	Visual Studio Code	N/A
Alterar categoria de produto existente	Funcional	Modificar dados de uma categoria já cadastrada	Manual	Visual Studio Code	N/A

Resultados obtidos



Cronograma de testes					
Responsável	Data de início prevista	Data de real	Desc. Resultado	Status	Conclusão
Eberth Rodrigues	02/09/25	01/09/25	Acesso permitido com credenciais corretas.	Concluída	Aprovado
Eberth Rodrigues	02/09/25	01/09/25	Mensagem de erro exibida ao digitar senha incorreta.	Concluída	Aprovado
Eberth Rodrigues	02/09/25	01/09/25	Formulário salvo com dados corretos.	Concluída	Reprovado
Eberth Rodrigues	02/09/25	01/09/25	Sistema bloqueia salvamento e exibe alerta.	Concluída	Aprovado
Eberth Rodrigues	05/09/25	01/09/25	Estoque atualizado com as 10 unidades adicionadas.	Concluída	Aprovado
Eberth Rodrigues	05/09/25	01/09/25	Estoque reduzido em 5 unidades com sucesso.	Concluída	Aprovado
Eberth Rodrigues	05/09/25	01/09/25	Sistema impede retirada e exibe mensagem de erro.	Concluída	Reprovado
Eberth Rodrigues	05/09/25	01/09/25	Valor do estoque exibido corretamente pelo nome.	Concluída	Reprovado
Eberth Rodrigues	08/09/25	01/09/25	Alterações salvas e exibidas corretamente.	Concluída	Aprovado
Eberth Rodrigues	08/09/25	01/09/25	Produto removido do banco de dados com sucesso.	Concluída	Aprovado
Eberth Rodrigues	08/09/25	01/09/25	Fornecedor cadastrado e salvo com sucesso.	Concluída	Aprovado
Eberth Rodrigues	08/09/25	01/09/25	Dados do fornecedor atualizados corretamente.	Concluída	Aprovado
Eberth Rodrigues	12/09/25	01/09/25	Categoria criada e listada com sucesso.	Concluída	Aprovado
Eberth Rodrigues	12/09/25	01/09/25	Alterações na categoria salvas com sucesso.	Concluída	Aprovado
Eberth Rodrigues	18/08/25	18/08/25	Categoria removida com sucesso; não aparece na lista.	Concluída	Aprovado
Eberth Rodrigues	18/08/25	18/08/25	Quantidade adicionada e registrada no banco de dados.	Concluída	Aprovado

Resultados obtidos



Resultados dos Testes

- Total de testes realizados: 60
- Testes aprovados: 39
- Testes reprovados: 21

Principais falhas identificadas:

- Funcionalidades que não atendem aos requisitos esperados
- Erros em validações de campos obrigatórios
- Problemas na geração de relatórios
- Inconsistências no controle de entrada/saída de hardware

Observação: As reprovações foram registradas conforme o cronograma de testes, permitindo correções ágeis e revalidação posterior.

CONCLUSÃO



A execução dos testes no sistema **TBIT Manager** permitiu identificar **falhas relevantes** e validar **funcionalidades essenciais**. Com **60 testes aplicados e 21 reprovações**, foi possível compreender os **pontos críticos** do sistema e **propor melhorias**. A automação com **Selenium** trouxe **agilidade e precisão**, reforçando a importância de integrar testes automatizados ao processo. O projeto consolidou o **aprendizado sobre planejamento, execução e análise de testes**, mostrando como essa etapa é decisiva para garantir a **qualidade de um software**.



Muito obrigado!